

MULTIMEDIAANALYZER

Audio and Video Content Analysis Tools

Software Engineering Assignment Report

Student project topic:

A system enabling upload, transcription, summarization, and AI analysis of audio/video files. The system saves results to a database and alerts the user upon finding specific entities.

Repository package:

/Users/erni/PSE/multimedia_analyzer

1. REQUIREMENTS ANALYSIS

1.1 Product Vision

MultimediaAnalyzer is a web-based application for authenticated users who need to analyze spoken content from audio and video. The platform supports media upload, asynchronous processing, transcript generation, summarization, named entity extraction, and detection of user-defined watchlist matches. Analysis results are stored in a persistent database and displayed in a responsive UI.

1.2 Actors

- Authenticated User
- Administrator / Evaluator
- Background Worker
- External AI services or local AI runtime (integration-ready)

1.3 User Stories

- US-01: As a user, I want to register so I can access my private workspace.
- US-02: As a user, I want to sign in securely so I can manage my data.
- US-03: As a user, I want to upload audio or video so the system can analyze it.
- US-04: As a user, I want the upload to process in the background so the UI remains responsive.
- US-05: As a user, I want a transcript of spoken content.
- US-06: As a user, I want a concise summary of the transcript.
- US-07: As a user, I want named entities extracted from the content.
- US-08: As a user, I want to define a watchlist of important entities.
- US-09: As a user, I want alerts when watched entities are detected.
- US-10: As a user, I want to see historical results in one dashboard.
- US-11: As an evaluator, I want a documented prototype and test artifacts.
- US-12: As an administrator, I want containerized deployment for repeatability.

1.4 Main Use Cases

- UC-01 Register user
- UC-02 Login user
- UC-03 Upload media file
- UC-04 Queue media analysis
- UC-05 Transcribe media
- UC-06 Summarize transcript
- UC-07 Extract named entities
- UC-08 Compare entities with watchlist
- UC-09 Save analysis results
- UC-10 View dashboard results
- UC-11 Update watchlist
- UC-12 Review alert matches

1.5 Functional Requirements

```
FR-01 The system shall support user registration and login.
FR-02 The system shall issue JWT tokens for authenticated access.
FR-03 The system shall allow authenticated users to upload media files.
FR-04 The system shall persist media metadata and outputs in PostgreSQL.
FR-05 The system shall process uploads asynchronously using Celery and Redis.
FR-06 The system shall support audio extraction from video files via FFmpeg.
FR-07 The system shall generate a transcript for each uploaded file.
FR-08 The system shall generate a summary for each transcript.
FR-09 The system shall extract named entities from the transcript.
FR-10 The system shall allow users to maintain a watchlist of entities.
FR-11 The system shall flag and store alert matches.
FR-12 The system shall display results in a browser dashboard.
```

1.6 Non-Functional Requirements

```
NFR-01 The system shall be containerized with Docker and Docker Compose.
NFR-02 The UI shall be responsive for desktop and mobile.
NFR-03 The system shall isolate user data by authenticated session.
NFR-04 The solution shall separate API, worker, database, and frontend roles.
NFR-05 The project shall include test design and specification artifacts.
```

2. REQUIREMENTS PRIORITIZATION (MoSCoW)

The assignment requested a prioritized requirements table with:
user story, use case, priority.

ID	User Story	Use Case
	Priority	
P-01	Register and sign in securely Must	UC-01, UC-02
P-02	Upload audio or video for analysis Must	UC-03
P-03	Process uploads asynchronously Must	UC-04
P-04	Produce transcript output Must	UC-05
P-05	Persist analysis results Must	UC-09
P-06	View results in dashboard Must	UC-10
P-07	Generate transcript summary Should	UC-06
P-08	Extract named entities Should	UC-07
P-09	Configure watchlist Should	UC-11
P-10	Highlight alert matches Should	UC-12
P-11	Deploy with Docker Compose Should	Deployment
P-12	Send external notifications Could	Extension
P-13	Filter historical records Could	Extension

P-14 Full AI runtime integration tests	Extension
P-15 Anonymous use without login	Out of scope
P-16 Won't	
+-----+-----+	
-----+-----+	

3. USE CASE DIAGRAM

Source file:
docs/assignment/use_case_diagram.mmd

- User starts with registration and login.
- User uploads media.
- The system queues analysis.
- Processing includes transcription and analysis.
- The system stores outputs.
- The user reviews the results and alert matches.
- Administrator manages deployment environment.

4. CONCEPTUAL MODEL (CLASS DIAGRAM)

Main classes:

- ```
2. MediaAsset
 - id
 - owner_id
 - original_filename
 - stored_path
 - media_type
 - status
 - task_id
 - transcript
 - summary
 - entities
 - alert_matches
 - error_message
 - created_at
```

- updated\_at
- 3. AnalysisPipeline
  - extract\_audio()
  - transcribe()
  - summarize()
  - extract\_entities()
  - detect\_alert\_matches()
- 4. AuthService
  - hash\_password()
  - verify\_password()
  - create\_access\_token()
  - get\_current\_user()
- 5. MediaTask
  - process\_media()

#### Relationships:

- One User owns many MediaAsset records.
- MediaTask updates MediaAsset state.
- MediaTask uses AnalysisPipeline.
- AuthService authenticates User access.

Source file:

docs/assignment/conceptual\_model\_class\_diagram.mmd

## 5. STATE DIAGRAMS

Two main state models are defined.

### 5.1 MediaAsset Lifecycle

```

queued -> processing -> completed
queued -> processing -> alerted
queued -> processing -> failed

```

Interpretation:

- A new uploaded file enters the queued state.
- The worker moves it to processing.
- If analysis finishes without watchlist matches, the result is completed.
- If watchlist matches exist, the result is alerted.
- If extraction or processing fails, the result is failed.

### 5.2 User Session Lifecycle

```

anonymous -> registered -> authenticated -> anonymous

```

Interpretation:

- A visitor starts anonymous.
- After account creation, the visitor becomes registered.
- After login, the visitor is authenticated.
- On logout, the user returns to anonymous state.

Source file:

docs/assignment/state\_diagrams.mmd

## 6. SEQUENCE DIAGRAM AND ACTIVITY DIAGRAMS

### 6.1 Sequence Diagram: Main Application Flow

Main sequence:

1. User logs in on the frontend.
2. Frontend calls the FastAPI login endpoint.
3. API validates the user against PostgreSQL and returns a JWT.
4. User uploads a media file.
5. Frontend posts the file to the API.
6. API stores media metadata and queues a Celery task.
7. Celery/Redis dispatches the task to the worker.
8. Worker processes the file and writes results to PostgreSQL.
9. User refreshes the dashboard.
10. Frontend fetches stored results from the API.

Source file:

docs/assignment/sequence\_main\_process.mmd

### 6.2 Activity Diagram: Upload and Analysis

This activity diagram models the full ingestion and processing pipeline:

- User selects file
- Authentication is validated
- File is uploaded and stored
- MediaAsset record is created
- Worker is queued
- Video may be converted to audio
- Transcript is generated
- Summary and entities are generated
- Watchlist comparison occurs
- Results are persisted

Source file:

docs/assignment/activity\_upload\_and\_analysis.mmd

### 6.3 Activity Diagram: Watchlist Alerting

This activity diagram models the alert configuration and matching logic:

- User opens watchlist settings
- User enters entities
- API validates the session
- Watchlist is stored
- Worker reads watchlist during media processing
- Detected entities are compared
- Alerted or completed state is assigned

Source file:

docs/assignment/activity\_watchlist\_alerting.mmd

## 7. UI DESIGN

The implemented UI prototype is a modern dashboard layout with:

- Hero summary panel

- Live status / control panel
- Authentication card
- Upload card
- Watchlist editor
- Results feed

Design principles:

- Clear visual hierarchy
- Responsive layout
- Distinct action zones
- Readable analytics summaries
- Fast access to transcript, entities, and alerts

UI design specification source:  
docs/assignment/ui\_design\_spec.md

Practical note:

The running application itself acts as the prototype UI that can be shown during evaluation. Additional screenshots can be exported from the live application for inclusion in a final submitted PDF if needed.

## 8. TESTING STRATEGY AND TEST CASES

### 8.1 Strategy

The testing approach is based on use-case traceability.

Test layers:

- Unit tests for auth and media helper functions
- API tests for login, upload, and watchlist endpoints
- Integration tests for database persistence and background processing
- UI validation for the main user flow
- Acceptance testing against the assignment use cases

### 8.2 Core Test Cases

TC-01 Register New User

- Preconditions: email not present in database
- Steps: enter credentials, click Register
- Expected: account created, token issued, session active

TC-02 Login Existing User

- Preconditions: user exists
- Steps: enter valid credentials, click Sign In
- Expected: token returned, dashboard loads

TC-03 Upload Audio File

- Preconditions: authenticated session
- Steps: select audio file, queue processing
- Expected: file stored, media record created, processing starts

TC-04 Upload Video File

- Preconditions: authenticated session
- Steps: select video file, queue processing
- Expected: audio extraction path triggered, status progresses

TC-05 Update Watchlist

- Preconditions: authenticated session
- Steps: edit watchlist, save
- Expected: values stored and reloaded

#### TC-06 Alert Match Detected

- Preconditions: watchlist contains a detected entity
- Steps: upload file, wait, refresh dashboard
- Expected: status = alerted, alert\_matches populated

#### TC-07 Unauthorized Upload Attempt

- Preconditions: no active session
- Steps: attempt upload
- Expected: blocked in UI or rejected by API

#### TC-08 Invalid Login

- Preconditions: wrong password
- Steps: sign in with invalid credentials
- Expected: access denied, error displayed

Detailed source file:

docs/assignment/testing\_strategy\_and\_cases.md

Test report template:

docs/assignment/test\_report\_template.md

## 9. DELIVERY PACKAGE

The assignment requested delivery of all parts as a single PDF. This report is the consolidated single-file submission document produced from the repository artifacts and suitable for direct export.

Recommended submission artifact:

MultimediaAnalyzer\_Assignment\_Report.pdf

## 10. EXTRA POINTS (30p) - AI PROMPT FOR FUNCTION PROTOTYPE

An AI-oriented software specification prompt was created and integrated into the project to satisfy the extra-credit requirement for prompting an AI system to construct a function prototype of the application.

Prompt summary:

"Build a full-stack MVP called MultimediaAnalyzer. The system must support JWT authentication, media upload, background processing through Celery, PostgreSQL persistence, Redis broker usage, and a React/Next.js dashboard. For each uploaded audio or video file, the system should create a transcript, summary, named entities, and alert matches against a user-configured watchlist. Provide function prototypes and endpoint contracts..."

Source file:

docs/assignment/ai\_prototype\_prompt.md

## 11. EXTRA POINTS (10p) - PROTOTYPE TESTING AND TEST REPORT

The repository includes a test report template tied directly to the defined use cases and test cases. This supports evaluation of the created prototype against

the specification.

Report sections:

- Environment
- Test summary
- Detailed results by test case
- Evidence and notes

Source file:

docs/assignment/test\_report\_template.md

---

## 12. IMPLEMENTATION STATUS OF THE CURRENT PROTOTYPE

---

Implemented in the repository:

- FastAPI backend
- JWT authentication
- PostgreSQL persistence model
- Celery worker integration
- Redis broker integration
- FFmpeg-ready media processing path
- Responsive Next.js frontend
- Watchlist management
- Alert state storage
- Dashboard presentation of results

Current prototype note:

The repository is a working MVP structure. The AI processing service currently uses deterministic placeholder logic for transcription, summarization, and entity extraction so the end-to-end workflow can run consistently in the local development environment. The architecture is ready for real AI runtime integration (Whisper, OpenAI, spaCy, Transformers).

---

## 13. CONCLUSION

---

This assignment package defines the requirements, prioritization, UML-oriented models, UI design, testing strategy, and extra-credit AI specification for the MultimediaAnalyzer system. The application prototype and the assignment documentation are aligned: the developed system reflects the key use cases and provides a strong base for further expansion into a production-grade media analysis platform.

---

## APPENDIX A - SOURCE ARTIFACTS IN THE REPOSITORY

---

- docs/assignment/requirements\_analysis.md
- docs/assignment/prioritized\_requirements.csv
- docs/assignment/use\_case\_diagram.mmd
- docs/assignment/conceptual\_model\_class\_diagram.mmd
- docs/assignment/state\_diagrams.mmd
- docs/assignment/sequence\_main\_process.mmd
- docs/assignment/activity\_upload\_and\_analysis.mmd
- docs/assignment/activity\_watchlist\_alerting.mmd
- docs/assignment/ui\_design\_spec.md
- docs/assignment/testing\_strategy\_and\_cases.md



- docs/assignment/ai\_prototype\_prompt.md
- docs/assignment/test\_report\_template.md

=====  
END OF REPORT  
=====